Naive Bayes

Intro and Motivation

Naive Bayes is one of the oldest and most elegant probabilistic classifiers.

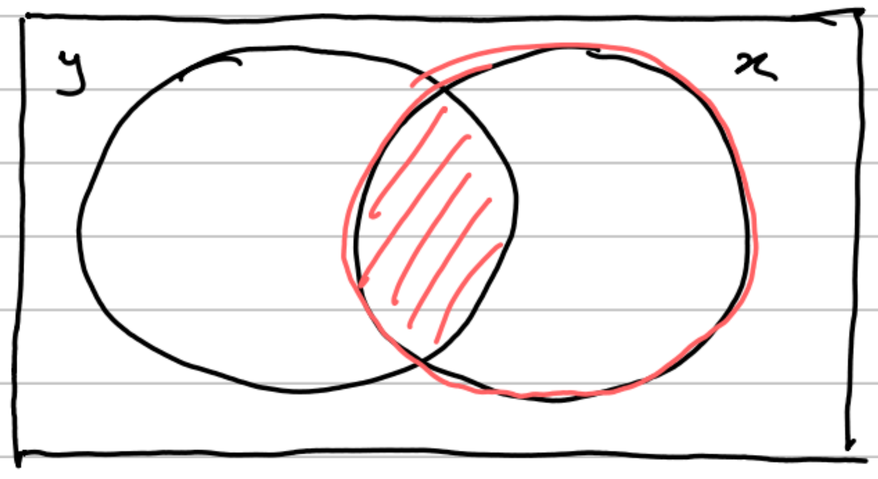
Naive Bayes is a generative model, as opposed to logistic regression, SVMs and decision trees, which are discriminative models. Discriminative models directly learn the boundary between classes: they model $p(y|x)$, the probability of the label given the features. Generative models instead model how the data was generated: they learn $p(x|y)$ the probability of the features given the label, and use Bayes' Theorem to recover $p(y|x)$.

Bayes' Theorem

Recall Bayes' Theorem:

$$p(y|x) = \frac{p(x|y) \cdot p(y)}{p(x)}$$

We can show this with a Venn Diagram:



$$p(y|x) = \frac{P(x \cap y)}{p(x)} \quad (1)$$

$$p(x|y) = \frac{P(x \cap y)}{p(y)} \quad (2)$$

∴ making $P(x \cap y)$ the subject and equating:

$$p(x)\,P(y|x) = p(y)\,p(x|y)$$

$$\therefore \quad p(y|x) = \frac{p(x|y) \cdot p(y)}{p(x)}$$

So why is this formulation useful? Bayes' Theorem is useful if you know the wrong conditional, when you want $P(A|B)$ but you can only directly measure or model $P(B|A)$. In classification, this is exactly the situation:

- you want $p(y|x)$ - the class given the features. This is what you need to make a prediction
- However, what you can naturally model is $p(x|y)$ - what features look like when you already know the class (training data).

Why is $P(x|y)$ easier to model? Because you can look at all training examples of class k and directly estimate what their features look like - their mean, variance etc.

Estimating $P(y|x)$ is harder, it requires drawing a complex boundary in feature space that accounts for all the ways classes overlap, without any explicit model of what the data looks like.

Imagine you're a doctor trying to diagnose whether a patient has a disease given their symptoms. You want $P(\text{disease}|\text{symptoms})$. What's easier to study? You look at patients with the disease and record what symptoms they have, $P(\text{symptoms}|\text{disease})$. You also look at healthy patients: $P(\text{symptoms}|\text{no disease})$, and you know the base rate $p(\text{disease})$ from population statistics. Bayes's Theorem lets you combine these three things you can measure to answer the question you actually want.

Each term has a name and role:

- $p(y|x)$
  The Posterior. The probability of class y given the observed features x. This is what we want for classification.

- $p(x|y)$
  The likelihood: how probable are these features given that the class is y? This is the generative model - it describes how the data was generated.

- $p(y)$
  The prior: how common is class y before seeing any features? This is estimated from the training label frequencies.

- $p(x)$
  The evidence: The overall probability of observing x under the model. It is the same for all classes, so it acts as a normalising constant and can be ignored for classification.

Classification via Bayes's Theorem

To classify a new point x, we compute the posterior ($p(y|x)$) for each class and pick the most probable one.

$$\hat{y} = \underset{y}{\operatorname{argmax}} \; p(y|x)$$

$$= \underset{y}{\operatorname{argmax}} \; p(x|y) \cdot p(y)$$

Since $p(x)$ is the same for all classes, it cancels in the argmax.

$p(y)$ is straightforward to calculate, it is the fraction of training examples belonging to class y. For a dataset with n_k examples of class k and n total examples. $p(y=k) = n_k/n$.

The challenge is estimating $p(x|y)$, the likelihood.

The Naive Conditional Independence Assumption

To estimate $p(x|y)$, x is a single data point and y is a label. We are asking, what is the probability of seeing this particular feature vector x, given that the class is y? We run into a problem as we try this.

x, a single example, consists of d features. So:

$$p(x|y) = p([x_1, x_2, \ldots x_d] | y).$$

To estimate this, we need to know the full joint distribution of d parameters conditioned on y. This means we must assign a probability to every possible combination of feature value and label. Once we have this, we can make full use of Bayes' theorem.

Let's use an example to illustrate this. Suppose We're classifying fruit as apple or orange. Two features:

- $x_1 \in \{0, 1\}$: is it red?
- $x_2 \in \{0, 1\}$: is it round?

Two classes:

$y \in \{\text{apple}, \text{orange}\}$

We want to find the joint distribution $P(x \cap y)$. Every possible $p(\text{feature vector} \cap \text{label})$ pair. If we have this, we can use it to find our likelihoods via:

$$p(x|y) = \frac{p(x \cap y)}{p(y)}$$

which we can then use in Bayes' Theorem.

Since we have 2^2 possible feature vectors and 2 possible labels, there are

$$2^2 \times 2 = 8$$

possible combinations. Suppose from our training data we estimate:

label \ features	00	01	10	11
apple	0.05 It's not red and not round, but it's an apple. Rare.	0.05 Not red, is round and is an apple. Rare.	0.10 It's red but not round. Is an apple. Less common	0.30 Red and Round apple. Common
Orange	0.10 Not red, not round orange. Less common	0.25 not red, round orange. Common	0.10 red, not round orange. Not common	0.05 Red and Round Orange. Rare.

All 8 sum to one. So we now have the joint, a description of how fruit features and labels co-occur. We use this to inform the question: given we know what fruit it is, how likely does it have x features? If we know it is an apple, what are the chances it is red and round?

We can now find the priors:

$$p(\text{apple}) = 0.05 + 0.05 + 0.10 + 0.30 = 0.50$$

$$p(\text{orange}) = 0.10 + 0.25 + 0.10 + 0.05 = 0.50$$

Equal priors - half the fruit are apples, half oranges. We can derive the likelihoods:

For a red, round fruit $(x = (1,1))$

$$p(x = (1,1) \mid \text{apple}) = \frac{P((1,1) \cap \text{apple})}{P(\text{apple})} = \frac{0.30}{0.50} = 0.6$$

$$p(x = (1,1) \mid \text{orange}) = \frac{P((1,1) \cap \text{orange})}{P(\text{orange})} = \frac{0.05}{0.50} = 0.1$$

Given it's an apple, 60% chance it's red and round. Given it's an orange, only 10% chance

Now we can drive the posterior via Bayes' Theorem: A new fruit arrives: red and round $x = (1,1)$. Which class is it? First, find the marginal $p(x = (1,1))$ - total probability of seeing a red round fruit:

$$p(x = (1,1)) = p((1,1), \text{apple}) + p((1,1), \text{orange})$$
$$= 0.30 + 0.05$$
$$= 0.35$$

Now use Bayes':

$$p(\text{apple}, (1,1)) = \frac{P((1,1) \cap \text{apple})}{p(x = (1,1))} = \frac{0.30}{0.35} \approx 0.857$$

$$p(\text{orange}, (1,1)) = \frac{P((1,1) \cap \text{orange})}{p(x = (1,1))} = \frac{0.05}{0.35} = 0.143$$

An 86% chance it's an apple. We predict apple.

Back To Naive Bayes'

Do you see the problem with trying to scale this up? Suppose we have d binary features and k classes. This requires $2^d - 1$ parameters per class - exponential in d. For d = 100 features and k = 2 classes, this is $2^{100} \approx 10^{30}$ parameters. Completely infeasible.

The Assumption

Naive Bayes assumes that features are conditionally independent, given the class label. That is to say, given that we know the class y, knowing the value of feature i tells us nothing additional about feature j.

Formally:

$$p(x_j \mid y, x_1, \dots x_{j-1}, x_{j+1}, \dots x_d)$$

$$= p(x_j \mid y)$$

Under this assumption, the joint likelihood factorises into a product of individual feature likelihoods:

$$p(x|y) = p(x_1, x_2, \dots, x_d \mid y)$$

$$= \prod_j p(x_j|y)$$

This reduces the number of parameters to linear in d - we only need to estimate one distribution per feature per class, rather than the full joint. With d features and K classes, we need $O(dK)$ parameters regardless of the feature space.

Why is it called Naive?

It is called naive because the conditional independence assumption is almost never literally true in real data.. Features are almost always correlated, the model ignores all of this structure.

However, it remarkably often works well anyway, for two reasons: For classification, we only need to get the argmax right - we do not need perfectly calibrated probabilities. Even if the absolute probability values are wrong due to violated independence, the ranking of classes may still be correct. Second, the conditional independence assumption provides strong regularisation - it prevents overfitting by limiting the model's capacity, which can be beneficial when data is limited.

The Full Model

Combining Bayes' theorem with the conditional independence assumption, the prediction is:

$$\hat{y} = \underset{y}{\operatorname{argmax}} \; p(y) \cdot \prod_j p(x_j \mid y)$$

We need to estimate two sets of parameters from the training data: the class prior $p(y)$ and the feature likelihoods $p(x_j|y)$ for each feature j and class y. Both are estimated by maximum likelihood from the training data.

The prior $p(y=k)$ is simply the fraction of training examples belonging to class k:

$$p(y=k) = \frac{n_k}{n}$$

The Log-Probability Trick

There is a critical numerical issue to address. $\prod_j p(x_j|y)$ multiplies d probabilities together. Each probability is between 0 and 1. For large d (e.g $d = 10,000$), this product underflows to zero in floating point arithmetic. The standard fix is to take the log of the entire expression. Since log is monotonically increasing, it doesn't change the argmax.

$$\hat{y} = \underset{y}{\operatorname{argmax}} \left(\log(p(y)) + \sum_j \log(p(x_j \mid y)) \right)$$

Products become sums, probabilities become log-probabilities, and underflow is eliminated.